AI-Powered Fake News Detection Using Text Classification

Snigdha Routray

***Abstract*—The rapid growth of digital media has significantly increased the spread of misinformation through online news platforms and social media. Detecting fake news manually has become increasingly difficult because of the enormous volume of information generated every day. This project presents a complete machine learning pipeline for fake news detection using Natural Language Processing (NLP) techniques and supervised machine learning algorithms. The dataset is preprocessed through text cleaning, tokenization, stopword removal, and feature extraction using Bag-of-Words, TF-IDF, and Word2Vec embeddings. Four machine learning models, namely K-Nearest Neighbors (KNN), Logistic Regression, Random Forest, and a Multi-Layer Perceptron (MLP) Neural Network, are trained and evaluated. The models are compared using accuracy, precision, recall, F1-score, and confusion matrices to identify the most effective classifier for fake news detection.**

***Index Terms*—Fake News Detection, Natural Language Processing, Machine Learning, Text Classification, TF-IDF, Bag-of-Words, Word2Vec, Random Forest, Logistic Regression, Neural Networks**

I. Introduction

A. Problem Statement

The emergence of digital media and social platforms has made the publication of news easy, resulting in the increase in misinformation and news articles. It is now very difficult to differentiate between genuine news and fake news articles. The proposed project aims at developing a system to classify an input news article as either real or fake, without using an existing end-to-end automated system to detect fake news articles.

B. Importance of Fake News Detection

Fake news can affect the opinion of people, impact the election process, cause civil unrest, and even damage genuine journalism. Manual fact-checking cannot scale in today's world where there is huge amount of information being generated every day through news and social media platforms. Therefore, an automated text classification system could help detect such suspicious information which will be flagged for human verification. This makes the proposed problem interesting and practical to solve using NLP & ML.

C. Objective

The objective of this project is to develop a complete machine learning pipeline from text preprocessing, feature extraction, classification, all the way to comparing the performance of different machine learning classifiers.

II. Dataset Description

A. Source

The dataset used is the ISOT Fake News Dataset, taken from Kaggle: "Fake News Detection Datasets" by Emine Bozkus (emineyetm/fake-news-detection-datasets).

The original dataset has been compiled from real world data where factual news articles have been scraped from Reuters.com and fake news articles from several untrustworthy websites, including those marked as fake by websites like Politifact and Wikipedia.

B. Size and Composition

The dataset is provided as two separate CSV files:

TABLE I
Dataset Composition

File	Articles	Source
True.csv	21,417	Reuters.com
Fake.csv	23,481	Various unreliable outlets

After combining the two files, removing rows with missing values, and dropping exact duplicate articles, the final working dataset contained 38,646 articles, with a class distribution of:

- Real (label = 1): 21,192 articles
- Fake (label = 0): 17,454 articles

This is a fairly balanced binary classification problem (approximately 55% real/45% fake), and no resampling methods, such as SMOTE or class weighting, are needed.

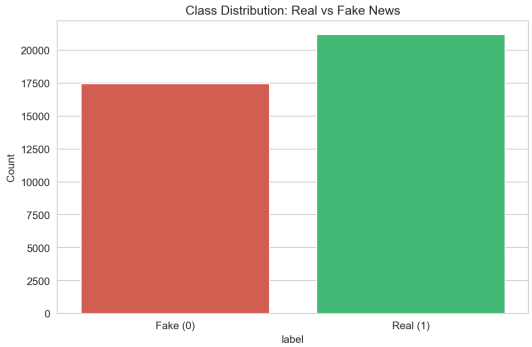

Fig. 1. Class Distribution of Real and Fake News Articles

C. Features

Each raw article record contains four fields: title, text, subject, and date. For this project, the title and text fields were concatenated into a single full_text field, which served as the sole input to the model pipeline. The subject and date fields were excluded from modeling, since subject in particular is strongly correlated with the source and would leak the label rather than reflect genuine linguistic signal.

D. Data Quality

Data quality review showed that:

- No missing values in any relevant column
- No duplicates found after preprocessing
- All features have correct type (object in case of text fields, int64 in case of label)

E. Resolving Known Leakage in the Dataset

While carefully examining the raw dataset, it turned out that almost all articles from Reuters start with a certain format feature, which is the dateline of "CITY (Reuters) -" format that is not used by the fake news. Failing to address the issue, only one piece of information will allow the classifier to gain a very high accuracy score without identifying any specific feature of fake news. The prefix of Reuters dateline was removed from every real article before the feature extraction, thus all algorithms had to use actual article content. This step and its implications on accuracy results are discussed in Discussion.

III. Methodology

A. Data Preprocessing

Preprocessing of texts was done manually (without using existing NLP libraries like NLTK or spaCy in the actual text cleansing procedure) and included the following steps performed sequentially for the combined title+text column for each article:

1) **Lowercasing:** conversion of all text into lowercase to ensure case insensitivity.
2) **Removal of URLs and HTML tags:** regular expression used to remove hyperlinks and all HTML tags.
3) **Removal of punctuation and digits:** all non-alphabetical characters replaced by spaces.
4) **Space normalization:** repeated spaces collapsed to one space and trimming.
5) **Tokenization:** splitting of cleansed text into tokens via whitespace separation (manual function without library tokenizer usage).
6) **Stopwords removal:** hardcoded English stopwords list (containing function words like "the," "is," "and," etc.) used for manual filtering of tokens; also tokens of length ≤ 1 filtered out.

Cleansing process took about 9.2 seconds for all 38,646 articles.

Example transformation:

Raw: "TOP 10 TWEETS From Democrat Debate — Here's the fake black guy, Shaun King weighing..."

Cleaned: "top tweets democrat debate fake black guy shau..."

B. Exploratory Data Analysis

An exploratory data analysis was conducted to examine class imbalance, article length, and word usage differences in real versus fake news articles.

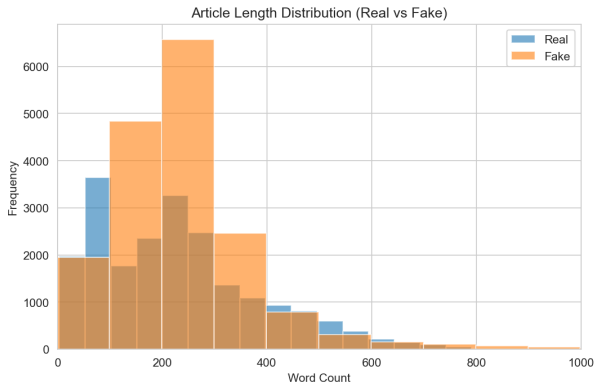

Fig. 2. Length Distribution of Articles

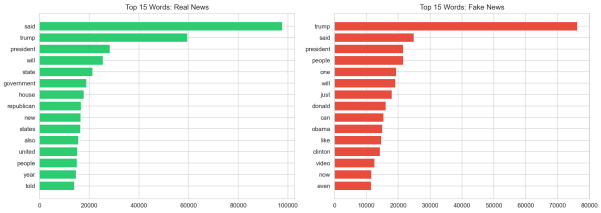

Fig. 3. Top 15 Words in Real and Fake News Articles

The differences in word usage patterns in the two classes were evident, particularly the mention of political figures and politics in the articles.

C. Feature Extraction

Three feature representation methods were employed to transform the cleaned text data into numeric data for machine learning:

1) **Bag-of-Words (BoW)** – represented using a count-based vectorizer constrained to only use the top 5,000 terms in the corpus, resulting in a sparse matrix of dimensions (38,646 $\times$ 5,000).
2) **TF-IDF (Term Frequency–Inverse Document Frequency)** – represented through two ways, either using a pre-defined vectorizer (unigrams + bigrams; limited to 5,000 features; resulting in a matrix of dimensions (38,646 $\times$ 5,000)) or through building the algorithm from scratch and computing the term frequency-inverse document frequency manually in order to understand the algorithm. TF-IDF representation of the corpus was used for all further model training, as it down-weights the commonly occurring terms in the entire corpus and focuses on the discriminative terms.

3) **Word Embeddings (Word2Vec)** – a Word2Vec model was also constructed using the tokenized corpus (vector size = 100; context window = 5; min. word count = 2); resulting in a 100-dimensional dense vector representation of each word in the corpus. Vocabulary size was equal to 66,164 terms. This approach was taken to learn distributed word representations as an alternative to sparse BoW/TF-IDF feature representation.

D. Implemented Algorithms

Four classification algorithms, ranging from both parametric and non-parametric model classes, were used to train the TF-IDF features:

1) **K-Nearest Neighbors (KNN)** – a non-parametric, instance-based classifier (k=5) which classifies the document by considering the predominant label among the k-nearest neighbors.
2) **Logistic Regression** – a parametric linear classifier which uses the linear model to determine the log-odds of the "real" class from the TF-IDF features (max iterations = 1,000).
3) **Random Forest** – a non-parametric ensemble algorithm containing 100 decision trees where each tree is built on a bootstrapped dataset, using the majority voting to determine the prediction.
4) **Multi-layer Perceptron (Neural Network)** – a parametric feed-forward neural network having one hidden layer containing 100 neurons, training up to 300 iterations.

All algorithms were trained using the same random seed number (42).

IV. Results

A. Training Performance

TABLE II
TRAINING PERFORMANCE

Model	Training Time (s)
KNN	0.0
Logistic Regression	0.3
Random Forest	6.0
Neural Network	116.7

KNN needed very little time for training because it is a lazy learning algorithm. The Logistic Regression and Random Forest algorithms needed minimal training time because the features were sparse and low dimensional (TF-IDF). However, the Neural Networks needed more training time because it used weight optimization during multiple iterations.

B. Classification Metrics

TABLE III
CLASSIFICATION METRICS

Model	Accuracy	Precision	Recall	F1-Score
KNN	0.6846	0.9395	0.4541	0.6123
Logistic Regression	0.9840	0.9813	0.9896	0.9854
Random Forest	0.9849	0.9815	0.9910	0.9863
Neural Network	0.9889	0.9908	0.9889	0.9898

Neural Networks scored better in overall performance as well as in F1 score, slightly beating the second and third most effective classifiers Random Forest and Logistic Regression. The worst performing classifier was KNN due to much lower recall rate of 0.4541 – KNN correctly classified real news articles half of the time.

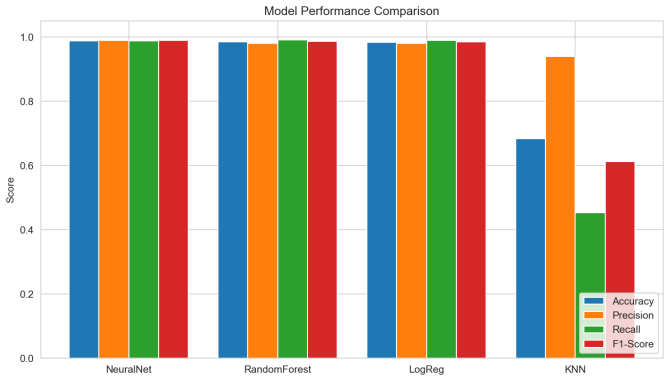

Fig. 4. Model Performance Comparison

C. Per-Class Performance

TABLE IV
PER-CLASS PERFORMANCE OF ALL MODELS

Model	Class	Precision	Recall	F1	Support
KNN	Fake	0.59	0.96	0.73	3491
KNN	Real	0.94	0.45	0.61	4239
Logistic Regression	Fake	0.99	0.98	0.98	3491
Logistic Regression	Real	0.98	0.99	0.99	4239
Random Forest	Fake	0.99	0.98	0.98	3491
Random Forest	Real	0.98	0.99	0.99	4239
Neural Network	Fake	0.99	0.99	0.99	3491
Neural Network	Real	0.99	0.99	0.99	4239

KNN exhibits significant asymmetry in that it is highly biased towards classification into "Fake" leading to high recall for fake data (0.96) and low recall for real data (0.45). The other three classifiers are equally good for both classes.

D. Confusion Matrices

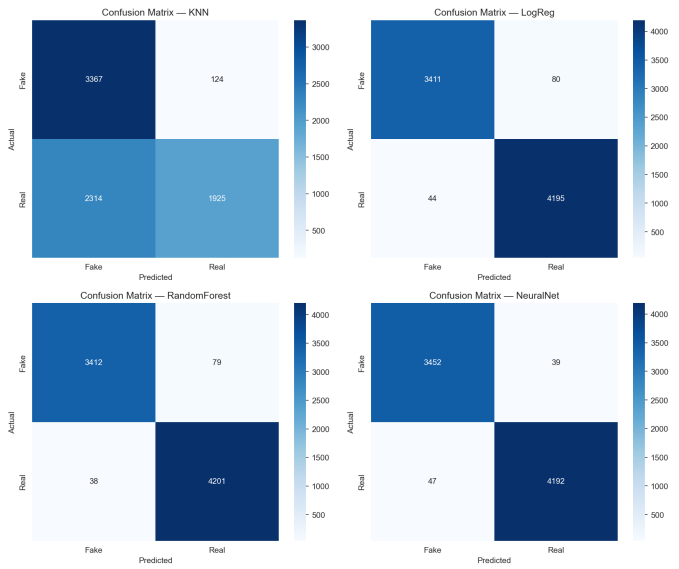

Fig. 5. Confusion Matrices for the Four Classification Models

The confusion matrices confirm the reports of the classifier: the Logistic Regression, the Random Forest, and the Neural Network have high concentrations on the diagonal without much misclassification in either direction, whereas the KNN classifies many actual articles as fake articles.

V. Discussion

A. Parametric vs. Non-Parametric Models

This study has deliberately used two parametric algorithms (Logistic Regression, Neural Network) and two non-parametric algorithms (KNN, Random Forest) to observe how each works for high dimensional and sparse text datasets.

TABLE V
Parametric and Non-Parametric Models

Category	Models
Parametric	Logistic Regression, Neural Network
Non-Parametric	KNN, Random Forest

The assumption of parametric models is the existence of a fixed functional form of the mapping with the fixed number of parameters to be trained. Both Logistic Regression and Neural Network performed very well on this task (98.4% and 98.9% accuracy correspondingly), which implies that the dependence between the values of the TF-IDF term weight vector and the real/fake label is almost linearly separable.

In turn, non-parametric models usually impose fewer restrictions on the form of the distribution in question and their complexity increases along with the increase in the size of the training sample. Random Forest, which is an ensemble of decision trees, performed equally well compared to the parametric approaches (98.5% accuracy) due to its robustness in dealing with non-linear feature interactions and the absence of the risk of overfitting because of the approach of bagging. On the other hand, KNN performed much worse (68.5% accuracy). It is consistent with the known problem of KNN, its poor performance in cases of high-dimensional (5,000 features) sparse input vectors due to the curse of dimensionality of the metric distance used in classification (e.g., Euclidean or cosine).

B. The Dataset Leakage Problem and Its Consequences

It is well-known that the ISOT dataset used in this project is plagued by the problem of leaking of confounding structural signals, namely the Reuters dateline prefix, which makes it possible for models to reach unrealistically high accuracies based not on learning any real markers of misinformation but by relying on these leaks. Although this particular leak was addressed in this project by removing the Reuters dateline prior to training, there is no doubt that other source-level leakage remains: real articles were taken from one newswire in consistent writing style, whereas fake articles came from several different news outlets in different writing styles and formats. As a result, the high accuracies (98–99% for three out of four models) reached by the models used in this project must also, at least partly, come from their ability to distinguish writing styles of Reuters from other sources, and not only from being able to distinguish factual truth from falsehood.

C. The Reasons Behind the Bad Performance of KNN

In addition to the problem related to the high dimensionality mentioned above, the low recall for the real news category indicates that the fake news category had more compact clusters in the TF-IDF space (this can be attributed to the greater vocabulary variety as well as to the presence of sensational language structures), making borderline real instances gravitate towards the majority class among their closest neighbors. This demonstrates a more general principle in relation to text classification.

D. Word Embeddings vs. Sparse Features

Whereas the main models built for this project used TF-IDF features, a Word2Vec embedding model was also developed on the corpus to experiment with the dense vector representation as another possible approach. Word2Vec models learn distributed representations which encode semantic relations among words using their co-occurrence context, while BoW and TF-IDF models use only frequency information. In future work, it would be interesting to experiment with averaging or pooling the Word2Vec vectors at the document level and comparing classification accuracy with the current TF-IDF approach.

VI. Conclusion

A. Lessons Learned

The present project showcased a full machine learning pipeline from scratch for fake news detection including manual text preprocessing, several feature extraction approaches

(BoW, TF-IDF and Word2Vec), and four different classification algorithms ranging between parametric and non-parametric models. The highest performance was delivered by the Neural Network (98.89% accuracy and 0.9898 F1-Score), followed by Random Forest and Logistic Regression, whereas KNN performed poorly due to the curse of dimensionality in sparse text features.

B. Limitations

1) The present data collection suffers from source-level confounds (different writing style between Reuters vs. other sources) which probably boosts the accuracy results above what would be possible to get for real fake news.
2) The dataset is limited to a certain period of time and only relates to US political news of 2016-2017, thus, it cannot be generalized to other topics, periods of time or even languages.
3) Only one hyperparameter setup of KNN (k=5) was tested, and the algorithm was not tuned systematically.
4) Word2Vec embeddings were trained, however, they were not used as inputs for classifiers in the current experiment.

C. Future Scope

1) Test the pipeline on a more diverse, multi-source, and temporally more recent data set to test the generalization performance in the face of new forms of misinformation.
2) Experiment with adding Word2Vec or other dense representations (GloVe, FastText etc.) in addition to TF-IDF in the classification pipeline.
3) Exploit transformer models such as BERT and RoBERTa for language understanding.
4) Conduct hyper-parameter optimization for various models (for example, tune the number of nearest neighbors in KNN model and regularization parameter in Logistic Regression).
5) Add metadata features like publication source credibility scores and social engagement metrics where possible, but cautiously to avoid re-labeling.

VII. Appendix

A. Sample Predictions on Test Data (Logistic Regression)

The Logistic Regression classifier was evaluated on the held-out test dataset. Representative predictions included correctly classified Real and Fake news articles, with only a small number of misclassifications observed. The sample predictions generated during testing are provided in the accompanying `test_data_samples.csv` file submitted with this project.

B. Dataset Summary

TABLE VI
Dataset Summary

Metric	Value
Total articles (after cleaning)	38,646
Real articles	21,192
Fake articles	17,454
Train set size	30,916
Test set size	7,730
TF-IDF feature dimension	5,000
Word2Vec vocabulary size	66,164

C. Setup and Imports

```
import pandas as pd
import numpy as np
import re
import string
import random
import time
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import
    TfidfVectorizer, CountVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score,
    classification_report, confusion_matrix
)

RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
```

D. Data Loading and Label Construction

```
true_df = pd.read_csv("True.csv")
fake_df = pd.read_csv("Fake.csv")

# Label: 1 = Real, 0 = Fake
true_df["label"] = 1
fake_df["label"] = 0

data = pd.concat([true_df, fake_df], axis=0,
    ignore_index=True)
data = data.dropna(subset=["title", "text"])
data = data.drop_duplicates(subset=["text"])
data = data.sample(frac=1, random_state=RANDOM_STATE
    ).reset_index(drop=True)

print("Combined dataset shape:", data.shape)
print("Class balance:\n", data["label"].value_counts
    ())
```

E. Removing Source-Specific Leakage

Real articles in the ISOT dataset are prefixed with a Reuters dateline (e.g., `"WASHINGTON (Reuters) -"`) that does not appear in fake articles. This artifact is removed to prevent

the classifier from learning a trivial formatting shortcut instead of genuine content-based signal.

```
def remove_reuters_tag(text):
    return re.sub(r'^.*?\(Reuters\)\s*-\s*', '',
    text)

data["text"] = data["text"].apply(remove_reuters_tag
    )
```

F. Manual Text Preprocessing

```
STOPWORDS = set("""
a about above after again against all am an and any
    are aren't as at be
because been before being below between both but by
    can't cannot could
couldn't did didn't do does doesn't doing don't down
     during each few for
from further had hadn't has hasn't have haven't
    having he he'd he'll he's
her here here's hers herself him himself his how how
    's i i'd i'll i'm i've
if in into is isn't it it's its itself let's me more
     most mustn't my myself
no nor not of off on once only or other ought our
    ours ourselves out over
own same shan't she she'd she'll she's should
    shouldn't so some such than
that that's the their theirs them themselves then
    there there's these they
they'd they'll they're they've this those through to
     too under until up
very was wasn't we we'd we'll we're we've were weren
    't what what's when
when's where where's which while who who's whom why
    why's with won't would
wouldn't you you'd you'll you're you've your yours
    yourself yourselves
""".split())

def clean_text(text):
    """Lowercase, strip URLs/HTML, remove
    punctuation & digits."""
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', ' ', text)
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'[^a-z\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def manual_tokenize(text):
    """Simple whitespace tokenizer (manual, no NLTK)
    ."""
    return text.split()

def remove_stopwords(tokens):
    return [t for t in tokens if t not in STOPWORDS
    and len(t) > 1]

def preprocess(text):
    text = clean_text(text)
    tokens = manual_tokenize(text)
    tokens = remove_stopwords(tokens)
    return " ".join(tokens)

data["full_text"] = data["title"] + " " + data["text
    "]
data["clean_text"] = data["full_text"].apply(
    preprocess)
```

G. Exploratory Data Analysis

```
# Class distribution
plt.figure()
sns.countplot(x="label", data=data, palette=["#
    E74C3C", "#2ECC71"])
plt.xticks([0, 1], ["Fake (0)", "Real (1)"])
plt.title("Class Distribution: Real vs Fake News")
plt.savefig("eda_class_balance.png", dpi=150,
    bbox_inches="tight")
plt.show()

# Article length distribution
data["word_count"] = data["clean_text"].apply(lambda
     x: len(x.split()))
plt.figure()
plt.hist(data.loc[data.label == 1, "word_count"],
    bins=50, alpha=0.6, label="Real")
plt.hist(data.loc[data.label == 0, "word_count"],
    bins=50, alpha=0.6, label="Fake")
plt.xlim(0, 1000)
plt.xlabel("Word Count")
plt.ylabel("Frequency")
plt.title("Article Length Distribution (Real vs Fake
    )")
plt.legend()
plt.savefig("eda_length_distribution.png", dpi=150,
    bbox_inches="tight")
plt.show()

# Most frequent words per class
from collections import Counter

def top_words(label_value, n=15):
    all_tokens = " ".join(
        data.loc[data.label == label_value, "
    clean_text"]
    ).split()
    return Counter(all_tokens).most_common(n)

real_top = top_words(1)
fake_top = top_words(0)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].barh([w for w, c in real_top][::-1],
             [c for w, c in real_top][::-1], color="
    #2ECC71")
axes[0].set_title("Top 15 Words: Real News")
axes[1].barh([w for w, c in fake_top][::-1],
             [c for w, c in fake_top][::-1], color="
    #E74C3C")
axes[1].set_title("Top 15 Words: Fake News")
plt.tight_layout()
plt.savefig("eda_top_words.png", dpi=150,
    bbox_inches="tight")
plt.show()
```

H. From-Scratch TF-IDF Implementation

```
def compute_tf(tokens):
    """Term frequency for one document."""
    tf = Counter(tokens)
    total = len(tokens)
    return {w: c / total for w, c in tf.items()} if
    total > 0 else {}

def compute_idf(tokenized_docs):
    """Inverse document frequency across the corpus.
    """
    n_docs = len(tokenized_docs)
    df = Counter()
    for tokens in tokenized_docs:
        for word in set(tokens):
```

```
            df[word] += 1
    return {w: np.log(n_docs / (1 + c)) + 1 for w, c
     in df.items()}

def compute_tfidf_manual(tokenized_docs):
    """Returns a list of {word: tfidf_score} dicts,
    one per document."""
    idf = compute_idf(tokenized_docs)
    tfidf_docs = []
    for tokens in tokenized_docs:
        tf = compute_tf(tokens)
        tfidf_docs.append(
            {w: tf_val * idf.get(w, 0) for w, tf_val
     in tf.items()}
        )
    return tfidf_docs, idf

sample_docs = data["clean_text"].iloc[:5].apply(
    manual_tokenize).tolist()
manual_scores, _ = compute_tfidf_manual(sample_docs)
```

I. Feature Extraction: BoW, TF-IDF, and Word Embeddings

```
X_text = data["clean_text"]
y = data["label"]

# Bag-of-Words
bow_vectorizer = CountVectorizer(max_features=5000)
X_bow = bow_vectorizer.fit_transform(X_text)

# TF-IDF (unigrams + bigrams)
tfidf_vectorizer = TfidfVectorizer(max_features
    =5000, ngram_range=(1, 2))
X_tfidf = tfidf_vectorizer.fit_transform(X_text)

print("Bag-of-Words shape:", X_bow.shape)
print("TF-IDF shape:", X_tfidf.shape)
```

```
from gensim.models import Word2Vec

sentences = [text.split() for text in data["
    clean_text"]]
word2vec_model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    seed=42
)
print("Vocabulary Size:", len(word2vec_model.wv))
```

J. Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

K. Model Training

```
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5,
    n_jobs=-1),
    "LogReg": LogisticRegression(max_iter=1000,
    random_state=RANDOM_STATE),
    "RandomForest": RandomForestClassifier(
        n_estimators=100, random_state=RANDOM_STATE,
     n_jobs=-1
    ),
    "NeuralNet": MLPClassifier(
        hidden_layer_sizes=(100,), max_iter=300,
    random_state=RANDOM_STATE
    )
}

trained_models = {}
training_times = {}

for name, model in models.items():
    t0 = time.time()
    model.fit(X_train, y_train)
    training_times[name] = time.time() - t0
    trained_models[name] = model
    print(f"{name} trained in {training_times[name
    ]:.1f}s")
```

L. Model Evaluation

```
results = []

for name, model in trained_models.items():
    preds = model.predict(X_test)

    acc = accuracy_score(y_test, preds)
    prec = precision_score(y_test, preds)
    rec = recall_score(y_test, preds)
    f1 = f1_score(y_test, preds)

    results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-Score": f1,
        "Train Time (s)": training_times[name]
    })

    print(f"\n{name}")
    print(classification_report(y_test, preds,
    target_names=["Fake", "Real"]))

results_df = pd.DataFrame(results).sort_values("
    Accuracy", ascending=False)
results_df.to_csv("model_results.csv", index=False)
```

M. Confusion Matrices and Model Comparison Plot

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for ax, (name, model) in zip(axes, trained_models.
    items()):
    preds = model.predict(X_test)
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues
    ", ax=ax,
                xticklabels=["Fake", "Real"],
    yticklabels=["Fake", "Real"])
    ax.set_title(f"Confusion Matrix: {name}")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.savefig("confusion_matrices.png", dpi=150,
    bbox_inches="tight")
plt.show()
```

```
metrics = ["Accuracy", "Precision", "Recall", "F1-
    Score"]
x = np.arange(len(results_df))
width = 0.2

fig, ax = plt.subplots(figsize=(11, 6))
for i, metric in enumerate(metrics):
    ax.bar(x + i * width, results_df[metric], width,
     label=metric)

ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(results_df["Model"])
ax.set_ylim(0, 1.05)
ax.set_ylabel("Score")
ax.set_title("Model Performance Comparison")
ax.legend(loc="lower right")
plt.savefig("model_comparison.png", dpi=150,
    bbox_inches="tight")
plt.show()
```

N. Parametric vs. Non-Parametric Classification

```
comparison = pd.DataFrame({
    "Category": ["Parametric", "Parametric",
                 "Non-Parametric", "Non-Parametric"
    ],
    "Model": ["Logistic Regression", "Neural Network
    ",
              "KNN", "Random Forest"]
})
```

O. Generating Test Data Samples for Appendix

```
test_sample = data.loc[y_test.index, ["title", "
    label"]].copy()
test_sample["predicted_LogReg"] = trained_models["
    LogReg"].predict(X_test)
test_sample = test_sample.sample(20, random_state=
    RANDOM_STATE)
test_sample.to_csv("test_data_samples.csv", index=
    False)
```